MËSIMI 26 / SHËNIME PËR INSTRUKTORIN

Projekti dhe komponenti Bird

75 minuta • App.js, Bird.js dhe Obstacles.js

Rezultati i pritshëm
Nxënësi krijon projektin bosh, ndërton Bird.js dhe e lidh me App.js. Në fund zogu katror bie poshtë, si në modulin origjinal.

Rrjedha e orës

Koha	Çfarë bën instruktori
0-5	Rikujtim: komponenti, props dhe useState.
5-15	Shpjego lojën dhe krijo/hap projektin.
15-30	Ndërto Bird.js si View statik dhe shfaqe në App.js.
30-45	Shpjego screenWidth, screenHeight, birdLeft dhe birdBottom.
45-60	Shto useEffect dhe setInterval për rënien.
60-70	Lidh props dhe zgjidh gabimet me nxënësit.
70-75	Pyetje kontrolli dhe ruajtja e punës.

Si ta përdorësh këtë PDF

Tre faqet e para janë shënime në shqip për drejtimin e orës. Më pas përdor materialin origjinal dhe kodin e tij në JavaScript. Jep një hap, provo rezultatin dhe vetëm pastaj vazhdo me hapin tjetër.

Ruaj të njëjtët emra variablash dhe të njëjtat props si në fotografitë e kodit. Shënimet shpjegojnë shembullin ekzistues; nuk shtojnë funksione të reja në lojë.

MËSIMI 26 / SHËNIME PËR INSTRUKTORIN

Komandat: ndryshimet e vetme teknike

Në materialin e vjetër	Përdor tani
expo init flappy-bird-rn --template blank	npx create-expo-app@latest flappy-bird-rn --template blank
npm run, për ta nisur projektin	npm start ose npx expo start

Rendi i komandave

```
npx create-expo-app@latest flappy-bird-rn --template blank
cd flappy-bird-rn
npm start
```

blank është template minimal në JavaScript. Mos zgjidh default ose blank-typescript për këtë seri. Për një projekt që ekziston, mos e krijo përsëri: hyr në dosjen e tij dhe ekzekuto vetëm npm start.

Struktura mbetet ajo e mësimit

```
flappy-bird-rn/
  App.js
  assets/
  src/
    components/
      Bird.js
      Obstacles.js  // krijohet ne modulin 27
```

Përgatitja para orës
Provoje nisjen dhe telefonin përpara klasës. Instalimet dhe rrjeti nuk duhet të të marrin pjesën e demonstrimit. Nëse Expo Go nuk e hap projektin, kontrollo SDK-në e mbështetur; ky është problem mjedisi, jo arsye për të ndryshuar kodin e mësimit.

Shpjegimi që mund t’u thuash nxënësve

"App.js mban vlerën e pozicionit. Bird.js e merr atë përmes props dhe vizaton katrorin. Kur ndryshon state, ndryshon edhe vendi ku shfaqet zogu."

MËSIMI 26 / SHËNIME PËR INSTRUKTORIN

Rënia: çfarë duhet të kuptojnë

Pjesa e kodit	Shpjegimi i thjeshtë
birdLeft	Pozicioni horizontal i zogut; vendoset në mes të ekranit.
birdBottom	Pozicioni vertikal i përdorur nga mësimi. Kur zvogëlohet, zogu zbret.
gravity = 3	Në këtë shembull zbriten 3 njësi në çdo hap; është vlerë e thjeshtë mësimore.
setInterval(..., 30)	Përsërit përditësimin afërsisht çdo 30 milisekonda.
clearInterval(gameTimerId)	Pastron intervalin që të mos mbetet aktiv pas cleanup-it.
Props te Bird	birdBottom dhe birdLeft duhet të kalojnë nga App.js te Bird.js.

Kontroll pas çdo hapi

1. Para intervalit: katrori shfaqet dhe nuk lëviz.
2. Pas intervalit: state ndryshon.
3. Pas lidhjes së props: katrori bie. Nëse state ndryshon por zogu jo, kontrollo props dhe emrat e tyre.

Pyetje për vlerësim

Pse përdorim state? Sepse ndryshimi duhet të shfaqet në ekran.
A duhet ta shkruajmë intervalin te Bird.js? Në këtë mësim ai mbetet në App.js.
Çfarë ndryshon kur gravity bëhet 1? Zogu zbret më ngadalë.

Burimi dhe komandat e verifikuara

Materiali bazë: Lesson 26, Digital School, PDF-ja e furnizuar. Pas këtyre 3 faqeve vijon materiali origjinal me numërtimin e vet. Kodi dhe rendi i hapave ruhen; në modulin 26 janë përditësuar vetëm komandat e krijimit dhe nisjes.

Template blank: JavaScript me strukturë minimale
https://docs.expo.dev/more/create-expo/

Nisja e projektit me Expo CLI
https://docs.expo.dev/more/expo-cli/

setInterval dhe clearInterval
https://reactnative.dev/docs/timers

Verifikuar më 31.10.2026. Provoje hapjen në pajisjen e klasës para orës; përputhshmëria e Expo Go varet nga SDK-ja e projektit.

Lesson 26 (Estimated time: 75 minutes)

Lesson 26

Project Setup & Bird Component

Contents:

- Flappy Bird

Notes for instructor:

Before starting to teach this lesson, you'll have to do a 5-minute review previous lesson to make sure they remember something from previous lessons. After you finish the review you should explain on the board the contents just to get them ready. All the coursework completed in class should be uploaded to GitHub.

Flappy Bird

In today's lesson, we start with Game Development, and as a game to develop, we selected Flappy Bird, one of the most simple-famous games. First, we need to demonstrate how the game will work if any of the students don't have information about how the game is played or how it works. For this, we use http://flappybird.io/, which demonstrates us in the best way. After explaining game logic to the students, we continue to set up a new project which will be **flappy-bird-rn.**

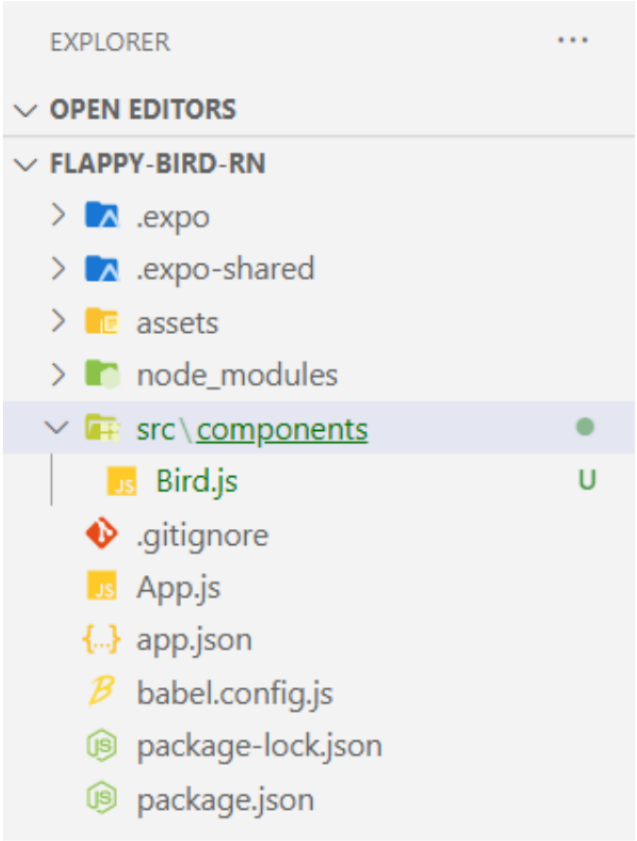
Somewhere on the Desktop, we open git bash, and we run the command which is used to create a project.

```
npx create-expo-app@latest flappy-bird-rn --template blank
```

After we're done with the installation process (dependencies & libraries), we type npm start. As always, this template will show us a message that is being rendered from the **App.js** file.

The game structure will be on components, so we're going to have **Bird** components, **Obstacles** components, etc. We're going to develop a simple game: the behavior of Bird and the behavior of Obstacles and movement and stuff.

We start by creating a folder that will be src inside of it. We will create a components folder as we did previously in other lessons the idea of this is to help students understand that the structure of the project is an important thing.



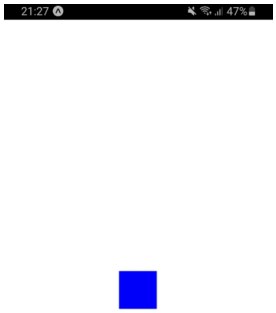
We continue to create the **Bird.js** component, which will be functional. We import required libraries and elements that we're going to use.

Bird.js App.js

src > components > Bird.js > Bird

```
1   import React from 'react';
2   import { View } from 'react-native';
3
4   const Bird = () => {
5       return (
6           <View style={{
7               position: 'absolute',
8               backgroundColor: 'blue',
9               width: 50,
10              height: 50
11          }}></View>
12      )
13  }
14
15  export default Bird;
```

Inside the Bird.js file, we create a View element with some inline style representing a blue box with fixed height & width. This will represent Bird by now since; first we're going to focus on the behavior, then later we're going to design the whole game. On the **App.js** file, we render the Bird Component, and we will get this view:



Making the Bird falling

To make this effect, we will use props and dynamic values, so first, we need to get the width and height of the screen. After that, we will need the left position and bottom position of the bird, so we declare two variables and store values on those.

App.js

```
import React, { useEffect, useState } from 'react';
import { StyleSheet, Text, View } from 'react-native';
import Bird from './src/components/Bird';

export default function App() {
  const screenWidth = Dimensions.get("screen").width
  const screenHeight = Dimensions.get("screen").height
  const birdLeft = screenWidth / 2
  const [birdBottom, setBirdBottom]= useState(screenHeight / 2)
  return (
    <View style={styles.container}>
      <Bird />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Now we start to make the bird's behavior falling by using useEffect Hook, which tells React that your component needs to do something after render.

App.js

App.js > App

```
import React, { useEffect, useState } from 'react';
import { StyleSheet, Text, View } from 'react-native';
import Bird from './src/components/Bird';

export default function App() {
  const screenWidth = Dimensions.get("screen").width
  const screenHeight = Dimensions.get("screen").height
  const birdLeft = screenWidth / 2
  const [birdBottom, setBirdBottom]= useState(screenHeight / 2)
  const gravity = 3
  let gameTimerId


  //Start bird falling
  useEffect(() => {
    if (birdBottom > 0) {
      gameTimerId = setInterval(() => {
        setBirdBottom(birdBottom => birdBottom - gravity)
      },30)

      return () => {
        clearInterval(gameTimerId)
      }
    }
    //If i dont have birdBottom as a dependecy, it wont stop
  }, [birdBottom])

  return (
    <View style={styles.container}>
      <Bird />
    </View>
  );
}
```

After we're done with the use Effect function, the Bird still will stay in that position, so for this to finish, we need to pass some props to the Bird Component.

We modify BirdComponent

App.js Bird.js

src > components > Bird.js > Bird

```
1   import React from 'react';
2   import { View } from 'react-native';
3
4   const Bird = ({birdBottom, birdLeft}) => {
5       const birdWidth = 50
6       const birdHeight = 60
7       return (
8           <View style={{
9               position: 'absolute',
10              backgroundColor: 'blue',
11              width: birdWidth,
12              height: birdHeight,
13              left: birdLeft - (birdWidth/2),
14              bottom: birdBottom - (birdHeight/2)
15          }}></View>
16      )
17  }
18
19  export default Bird;
```

After these modifications, when we open the app, we will see that our “Bird” is falling to the Zero point of the screen.

Source Code: Flappy Bird

Award the Fastest Student of React Native badge to the quickest student in this class: Award

Evaluation Criteria:

✓ All students should understand Bird Component

✓ Badges should be awarded

Tools needed:

✓ Computers

✓ VS Code

Jump to...

Copyright © Digital School. All rights reserved.